

Deusto

Universidad de Deusto

DISEÑO DE SOFTWARE

GRUPO Nº 13

INTEGRANTES

IKER GONZÁLEZ GARCÍA

ANDER GONZÁLEZ GARCÍA

ARITZ GONZÁLEZ SANTA CRUZ

OIER UNAMUNZAGA CAUJAPÉ

FECHA

11/11/2026

Índice

Leyenda y notación **3**
Supuestos de diseño **3**
Uso de la IA **4**
Diseño y arquitectura 4
Programación y herramientas 4
Pruebas y depuración 4
Documentación 4
Reflexión final 4
1. API REST del Servidor Ecoembes **5**
Convenciones generales 5
1.1. Autenticación 5
1.2. Gestión de contenedores 6
1.3. Lecturas y consultas 6
1.4. Plantas de reciclaje 7
1.5. Asignaciones y auditoría 8
2. Diagrama de clases **8**
Principales componentes 8
Diagrama de clases 9
3. Diagramas de secuencia **10**
3.1. Inicio de sesión, alta y consultas 10
3.2. Capacidad, asignación y auditoría 11
3.3. Cierre de sesión 12
3.4. Registro diario del sensor 12
4. Validación y pruebas **12**
4.1. Pruebas automatizadas con JUnit 12
4.2. Validación de la API REST 13
4.3. Validación mediante Swagger UI 13

Índice de operaciones REST

login 5
logout 5
createContainer 6
getContainers 6
getContainerReadings 6
getContainersByZone 6
registerReading 7
getPlants 7
getPlantCapacity 7
assignContainers 8
getAssignments 8

Leyenda y notación

Los colores identifican las distintas responsabilidades arquitectónicas del diagrama de clases.

La notación UML representa las relaciones entre sus elementos y las interacciones, mensajes y condiciones de los diagramas de secuencia.

DIAGRAMA DE CLASES

Façade	Paquetes
AppServices	Asociación y multiplicidad
StateManagement	Composición
Dominio	Dependencia
DTO y Assembler	Herencia y realización
Gateways	
Excepciones	

DIAGRAMAS DE SECUENCIA

participante	Líneas de vida
	Mensajes (peticiones y respuestas)
tipo	alt (alternativas) / opt (ejecución opcional) / break (interrupción del flujo)

Supuestos de diseño

Para este primer prototipo hemos establecido las siguientes decisiones y simplificaciones de diseño.

Estado en memoria Los datos se guardan en memoria y se restablecen al reiniciar el servidor.	**Autenticación** El personal utiliza tokens de sesión. Los sensores no necesitan autenticarse.	**Alta sin lectura** Los contenedores se crean sin mediciones hasta recibir su primera lectura.
Lectura diaria Solo se admite una lectura por contenedor y día. Las repetidas se rechazan.	**Historial de lecturas** Se guardan las mediciones y se puede consultar el estado en fechas anteriores.	**Peso y niveles** El sensor indica el nivel de llenado. Cada envase equivale a 0,03 kg.
Ventana de fechas Las capacidades y asignaciones se gestionan para los próximos diez días.	**Capacidad disponible** Se calcula restando las toneladas ya asignadas a la capacidad de cada planta.	**Asignación atómica** Si algún contenedor no cumple las condiciones, se rechaza toda la asignación.
Sin asignaciones duplicadas Un contenedor no puede asignarse a dos plantas el mismo día.	**Plantas simuladas** PlasSB y ContSocket funcionan mediante simulaciones, sin conexiones reales.	**Auditoría** Guardamos quién realiza cada asignación, cuándo y con qué cantidades.

Uso de la IA

Durante el desarrollo del Prototipo 1 de Ecoembes hemos utilizado **ChatGPT y Claude** para revisar nuestro trabajo, resolver dudas y ayudarnos con las pruebas y la documentación. Nos parecen herramientas muy útiles para contrastar ideas y conocer otras formas de afrontar un problema, aunque siempre hemos intentado mantener nuestro propio criterio.

Diseño y arquitectura

Empezamos diseñando por nuestra cuenta unos primeros diagramas UML y algunas operaciones de la API. Después recurrimos a **Claude** para mejorarlos, comprobar que no faltaba nada y conseguir una arquitectura sólida antes de programar.

Nos ayudó a descubrir formas de organizar el código que no habríamos planteado inicialmente, especialmente con los DTO y la gestión de errores. Eso sí, a veces complicaba demasiado las cosas y nos hacía perder tiempo revisando detalles poco importantes.

Programación y herramientas

El código lo hemos escrito nosotros, utilizando la IA para resolver dudas puntuales sobre sintaxis, librerías o Spring Boot. También nos ayudó a familiarizarnos con Git y GitHub.

Fue especialmente útil cuando nos atascábamos, aunque aprendimos que no conviene copiar soluciones o ejecutar comandos sin entender primero qué hacen.

Pruebas y depuración

Utilizamos **ChatGPT** para plantear casos de prueba, revisar resultados y detectar posibles fallos. Nos permitió comprobar situaciones que quizá no habríamos tenido en cuenta.

Sin embargo, algunas revisiones eran demasiado exigentes y convertían problemas menores en algo aparentemente grave. Por eso aprendimos a valorar mejor sus recomendaciones.

Documentación

La IA nos ayudó a redactar y organizar buena parte de la documentación a partir de nuestros diagramas y del proyecto desarrollado. Decidimos utilizarla porque escribir y dar formato a estos documentos nos resulta bastante más pesado que programar.

Nos ahorró mucho trabajo, pero también tuvimos que corregir explicaciones, eliminar repeticiones y ajustar el formato para que reflejara realmente lo que habíamos hecho.

Reflexión final

En general, la experiencia ha sido muy positiva. Hemos conocido soluciones más profesionales y aprendido nuevas formas de diseñar software.

También nos hemos dado cuenta de que **la IA no siempre tiene razón ni hace falta seguir todas sus recomendaciones**. Creemos que lo más importante ha sido partir de nuestras propias ideas, entender lo que desarrollamos y utilizar estas herramientas como apoyo, no como sustituto de nuestro trabajo.



1. API REST del Servidor Ecoembes

El servidor ofrece 11 operaciones REST para gestionar los contenedores, sus lecturas, las plantas de reciclaje y las asignaciones. También permite autenticar al personal y consultar el historial de operaciones realizadas.

Convenciones generales

ELEMENTO	DESCRIPCIÓN
Ruta base	/api
Acceso	Swagger UI: /swagger-ui/index.html
Formato	JSON para las peticiones y respuestas con cuerpo.
Datos	DTO (Data Transfer Object) como records Java.
Autenticación	Token numérico enviado como query parameter, salvo en el inicio de sesión y el registro de lecturas del sensor.
Fechas	Formato ISO yyyy-MM-dd.
Errores	Respuestas 400, 401, 404 y 409, según corresponda, mediante Error DTO.

1.1. Autenticación

login

POST /api/auth/login

PARÁMETROS Y ENTRADA	RETORNO	FUNCIONALIDAD
JSON LoginRequestDTO: email y password. No requiere token.	200 OK - LoginResponseDTO con el token de sesión.	Comprueba las credenciales del empleado y registra una sesión activa en el servidor.

Errores:
- **400** si faltan datos o son incorrectos en formato
- **401** si las credenciales no son válidas

logout

POST /api/auth/logout

PARÁMETROS Y ENTRADA	RETORNO	FUNCIONALIDAD
token (long) obligatorio como query parameter. Sin cuerpo.	204 No Content - Sin cuerpo de respuesta	Cierra la sesión asociada al token, impidiendo que vuelva a utilizarse.

Errores:
- **400** si el token falta o tiene un formato incorrecto
- **401** si no existe una sesión activa asociada

1.2. Gestión de contenedores

createContainer

POST /api/containers

PARÁMETROS Y ENTRADA	RETORNO	FUNCIONALIDAD
token obligatorio como query parameter. JSON ContainerCreateDTO: id, address, postalCode y capacity.	201 Created - ContainerDTO del contenedor creado.	Registra un nuevo contenedor, sin generar una lectura inicial. Los campos de medición permanecen a null.

Errores:
- **400** si los datos no son válidos
- **401** si no existe una sesión activa asociada al token
- **409** si ya existe un contenedor con ese identificador

getContainers

GET /api/containers

PARÁMETROS Y ENTRADA	RETORNO	FUNCIONALIDAD
token obligatorio como query parameter. Sin cuerpo.	200 OK - List<ContainerDTO>	Devuelve todos los contenedores con su última lectura disponible. Si alguno no tiene mediciones, sus campos correspondientes son null.

Errores:
- **400** si el token falta o tiene un formato incorrecto
- **401** si no existe una sesión activa asociada al token

1.3. Lecturas y consultas

getContainerReadings

GET /api/containers/{containerId}/readings

PARÁMETROS Y ENTRADA	RETORNO	FUNCIONALIDAD
containerId en la ruta. from, to (fechas ISO) y token como query parameters.	200 OK - List<ContainerReadingDTO>	Consulta las lecturas de un contenedor entre dos fechas, ambas incluidas, ordenadas cronológicamente.

Errores:
- **400** si falta el token o alguna fecha, tienen un formato incorrecto o from es posterior a to
- **401** si no existe una sesión activa asociada al token
- **404** si el contenedor no existe

getContainersByZone

GET /api/zones/{postalCode}/containers

PARÁMETROS Y ENTRADA	RETORNO	FUNCIONALIDAD
postalCode en la ruta. date (fecha ISO) y token como query parameters.	200 OK - List<ContainerDTO>	Consulta los contenedores de una zona utilizando su última lectura conocida hasta la fecha indicada.

Errores:
- **400** si el token, la fecha o el código postal faltan o tienen un formato incorrecto
- **401** si no existe una sesión activa asociada al token

Nota: Los contenedores sin lecturas disponibles hasta esa fecha no aparecen en el resultado.

registerReading

POST /api/containers/{containerId}/readings

PARÁMETROS Y ENTRADA	RETORNO	FUNCIONALIDAD
containerId en la ruta. JSON SensorReadingDTO: packageCount y fillLevel. No requiere token.	201 Created - ContainerReadingDTO	Registra una lectura del sensor utilizando la fecha actual del servidor.

Errores:
- **400** si los datos son inválidos o la cantidad supera la capacidad del contenedor
- **404** si el contenedor no existe
- **409** si ya existe una lectura registrada ese día

Nota: Solo se conserva la primera lectura de cada día; las posteriores nunca la sustituyen.

1.4. Plantas de reciclaje

getPlants

GET /api/plants

PARÁMETROS Y ENTRADA	RETORNO	FUNCIONALIDAD
token obligatorio como query parameter. Sin cuerpo.	200 OK - List<PlantDTO>	Devuelve las plantas de reciclaje disponibles: PlasSB Ltd. y ContSocket Ltd.

Errores:
- **400** si el token falta o tiene un formato incorrecto
- **401** si no existe una sesión activa asociada al token

getPlantCapacity

GET /api/plants/{plantId}/capacity

PARÁMETROS Y ENTRADA	RETORNO	FUNCIONALIDAD
plantId en la ruta. date (fecha ISO) y token como query parameters.	200 OK - PlantCapacityDTO	Consulta la capacidad total, las toneladas asignadas y la capacidad disponible de una planta para una fecha.

Errores:
- **400** si el token o la fecha faltan o tienen un formato incorrecto o si la fecha está fuera de los próximos 10 días
- **401** si no existe una sesión activa asociada al token
- **404** si la planta no existe

Nota: La capacidad disponible se calcula restando las toneladas asignadas a la capacidad total de la planta.

1.5. Asignaciones y auditoría

assignContainers

POST /api/plants/{plantId}/assignments

PARÁMETROS Y ENTRADA	RETORNO	FUNCIONALIDAD
plantId en la ruta y token en query parameters. JSON AssignmentRequestDTO: date y containerIds.	201 Created - AssignmentDTO	Asigna varios contenedores a una planta, registra los totales y el empleado responsable y notifica a la planta simulada.

Errores:
- **400** si falta algún dato, la fecha está fuera de los próximos diez días o la selección está vacía o contiene ids repetidos
- **401** si no existe una sesión activa asociada al token
- **404** si la planta o algún contenedor no existen
- **409** si algún contenedor ya está asignado, no tiene lecturas o no hay capacidad suficiente

Nota: La operación es atómica: si falla alguna validación, no se registra ninguna asignación.

getAssignments

GET /api/plants/{plantId}/assignments

PARÁMETROS Y ENTRADA	RETORNO	FUNCIONALIDAD
plantId en la ruta, token obligatorio y date (fecha ISO) opcional como query parameters.	200 OK - List<AssignmentDTO>	Consulta las asignaciones registradas de una planta, incluyendo las cantidades y el empleado responsable.

Errores:
- **400** si el token falta o tiene un formato incorrecto o si se indica una fecha con formato incorrecto
- **401** si no existe una sesión activa asociada al token
- **404** si la planta no existe

Nota: Si no se indica date, se devuelven todas las asignaciones de la planta. Si se indica, únicamente las de esa fecha.

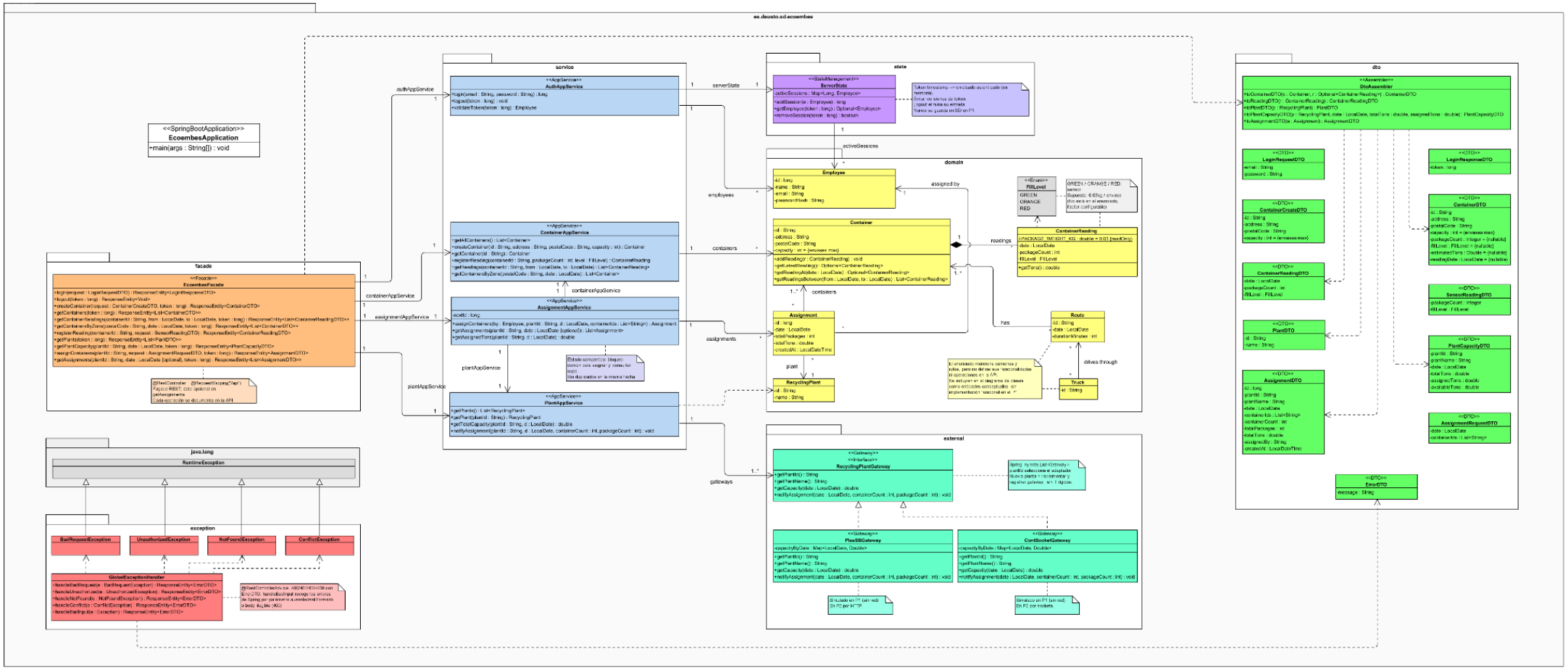
2. Diagrama de clases

El siguiente diagrama representa la estructura del Servidor Ecoembes y las relaciones entre sus clases. Su diseño sigue una arquitectura por capas, separando la gestión de peticiones REST, la lógica de negocio, los datos y las integraciones con las plantas de reciclaje.

Principales componentes

ELEMENTO	FUNCIÓN
Façade	EcoembesFacade recibe las peticiones REST y devuelve las respuestas HTTP.
AppService	AuthAppService, ContainerAppService, PlantAppService y AssignmentAppService gestionan los distintos casos de uso.
StateManagement	ServerState mantiene las sesiones activas del personal.
Dominio	Representa los contenedores, sus lecturas, las plantas y las asignaciones. Truck y Route se incluyen como conceptos, sin gestión funcional en este prototipo.
DTO	Define los datos intercambiados por la API, sin exponer directamente las entidades del dominio.
Gateways	Abstraen la comunicación con PlasSB Ltd y ContSocket Ltd, simulada en el prototipo.

Diagrama de clases



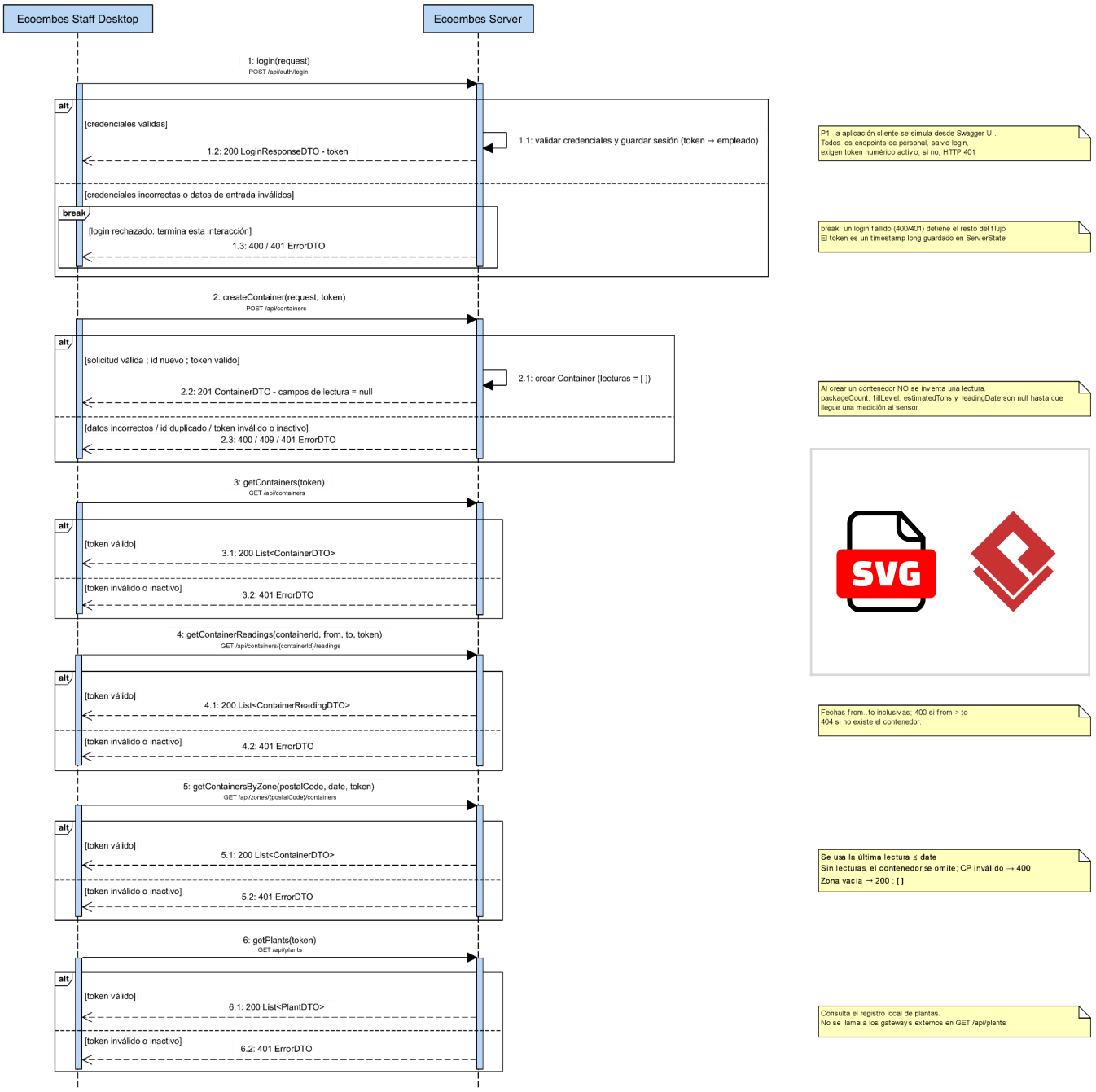
Debido a las limitaciones de Google Docs, no ha sido posible insertar el diagrama completo con la calidad deseada. Disculpamos la pérdida de nitidez y recomendamos descargarlo en formato **SVG** para visualizarlo en alta calidad vectorial.

3. Diagramas de secuencia

Los siguientes diagramas muestran cómo interactúan los distintos participantes del sistema durante las operaciones de la API. Se representan tanto los flujos correctos como las posibles situaciones de error, incluyendo la comunicación simulada con las plantas de reciclaje.

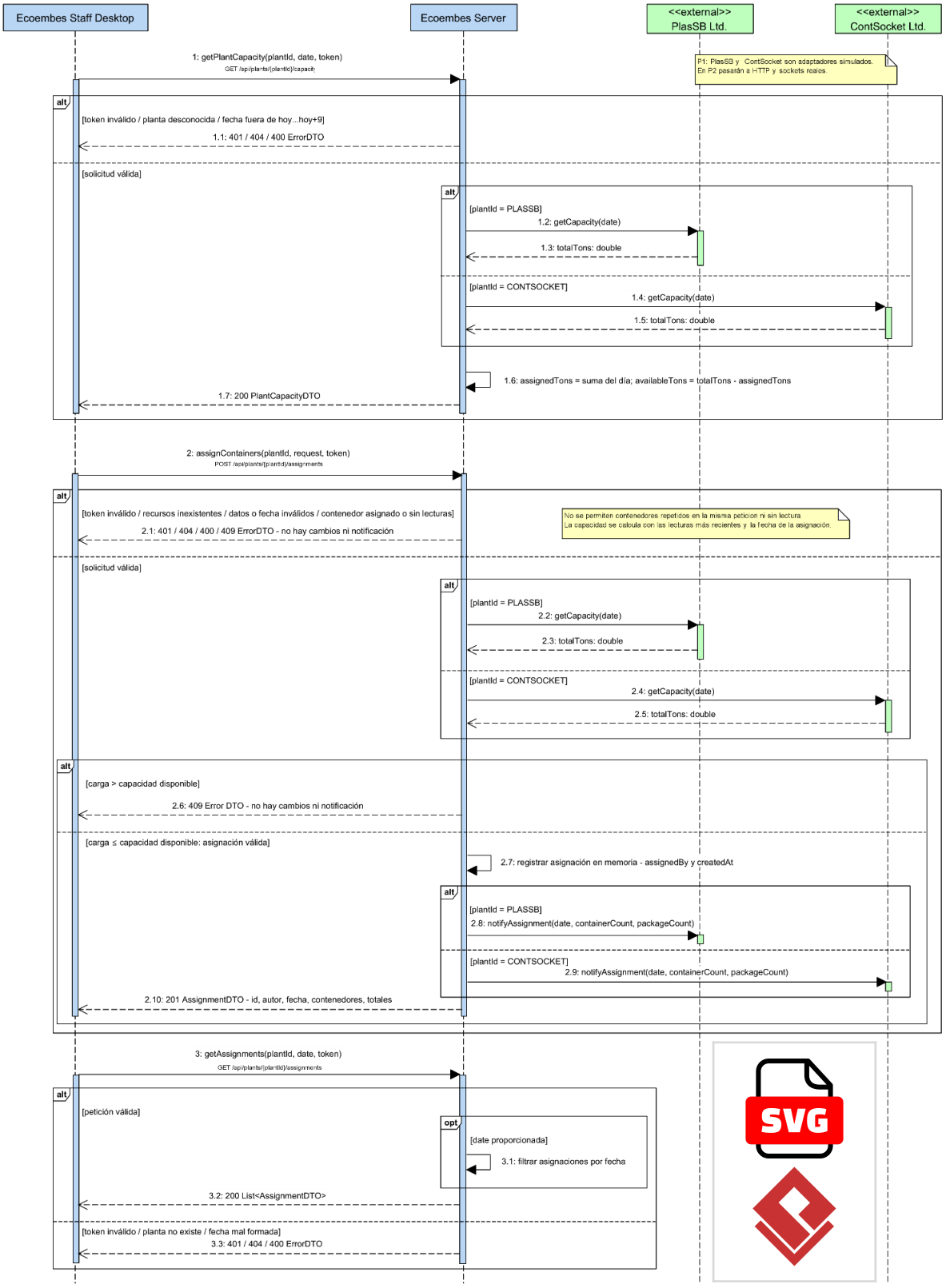
3.1. Inicio de sesión, alta y consultas

Este diagrama recoge el inicio de sesión, la creación de contenedores y las consultas de contenedores, lecturas, zonas y plantas. Si el inicio de sesión falla, la interacción se interrumpe; las demás operaciones del personal requieren una sesión válida.



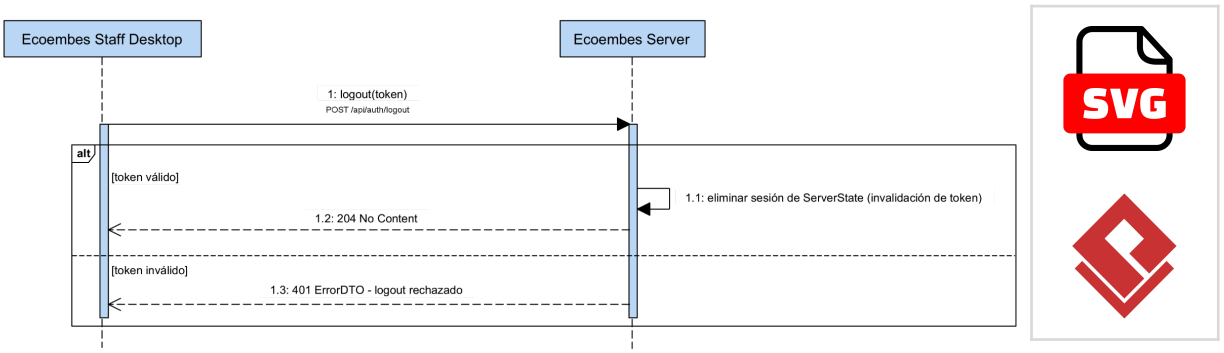
3.2. Capacidad, asignación y auditoría

Representa la consulta de capacidad, la asignación de contenedores y la consulta de asignaciones. Se incluyen las dos plantas de reciclaje como alternativas excluyentes, con sus comunicaciones simuladas. Las asignaciones solo se registran cuando se cumplen todas las condiciones y, después, se notifica a la planta correspondiente.



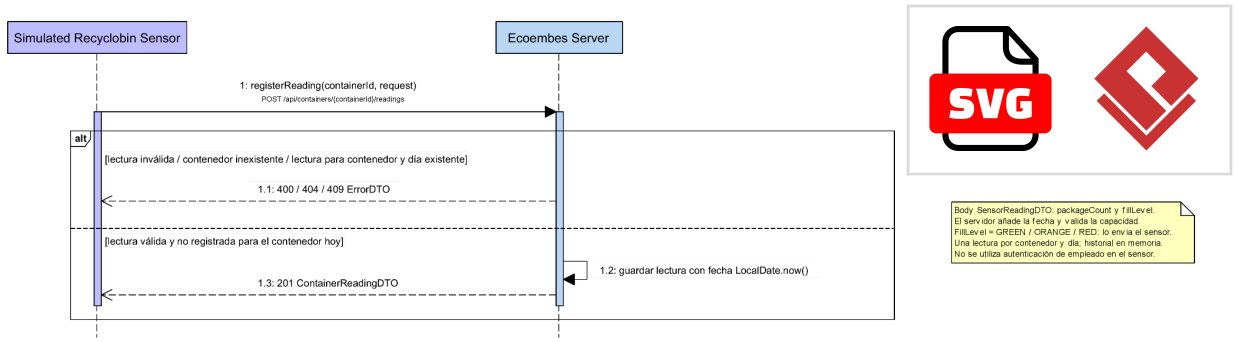
3.3. Cierre de sesión

Muestra el cierre de sesión del personal. Si el token está activo, se invalida y se devuelve 204 No Content. Si no existe una sesión asociada, el servidor responde con 401 Unauthorized.



3.4. Registro diario del sensor

Representa el envío de lecturas desde un sensor simulado, sin autenticación de personal. El servidor valida los datos y registra la primera lectura del día con 201 Created. Si ya existe una lectura para ese contenedor y fecha, devuelve 409 Conflict sin sustituirla.



4. Validación y pruebas

Para comprobar el funcionamiento del Servidor Ecoembes hemos realizado pruebas automatizadas con JUnit, validaciones HTTP y pruebas mediante Swagger UI. Hemos comprobado tanto el funcionamiento normal de las operaciones como las situaciones de error y sus efectos sobre los datos.

4.1. Pruebas automatizadas con JUnit

Las pruebas de JUnit han sido diseñadas con ayuda de IA, que nos ha permitido generar datos de prueba, plantear distintos escenarios y contemplar casos límite de las reglas de negocio. Esto nos ha ayudado a conseguir una cobertura amplia, sin tener que preparar manualmente todas las combinaciones de datos.

Las pruebas verifican la autenticación, los contenedores, las lecturas, la capacidad de las plantas, las asignaciones y el tratamiento de errores, incluyendo situaciones de concurrencia. Hemos revisado los casos propuestos para comprobar que se ajustaban al funcionamiento esperado.

Resultados: 63 pruebas distribuidas en 12 suites, todas superadas. La compilación completa mediante Gradle también finalizó correctamente.

4.2. Validación de la API REST

Además, hemos ejecutado 72 escenarios HTTP para comprobar las respuestas del servidor y el estado de los datos después de cada operación.

Se han probado situaciones como credenciales incorrectas, fechas inválidas, contenedores inexistentes, lecturas duplicadas o asignaciones sin capacidad suficiente. También hemos comprobado que las operaciones rechazadas no modifican los datos existentes.

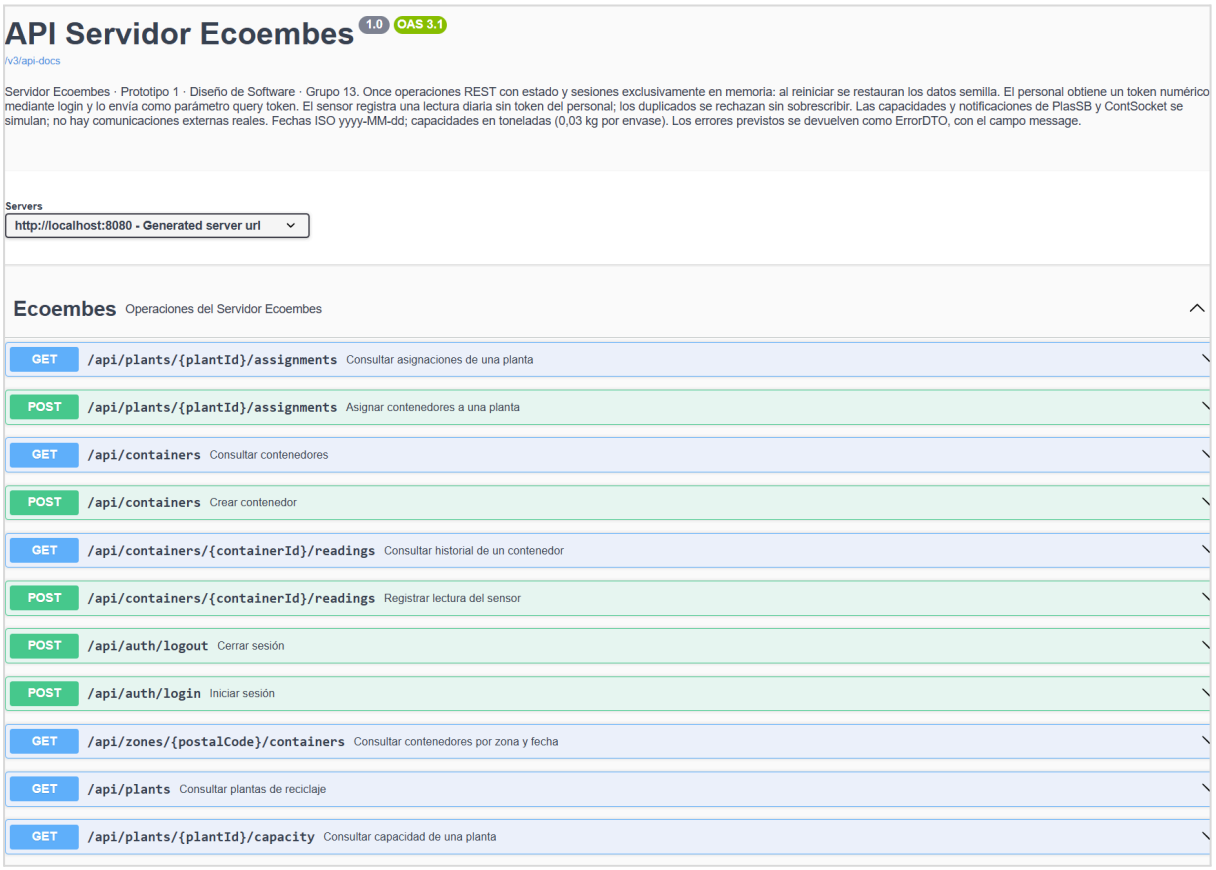
Resultados: 72 escenarios correctos, sin fallos registrados.

4.3. Validación mediante Swagger UI

Finalmente, hemos utilizado Swagger UI para ejecutar las 11 operaciones de la API y comprobar directamente sus respuestas.

Se realizaron 38 peticiones con los resultados esperados y se obtuvieron 39 capturas, incluyendo la vista general de Swagger. A continuación, mostramos las evidencias más representativas.

4.3.1. Vista general de la API



Vista general de las 11 operaciones disponibles en Swagger UI

4.3.2. Autenticación

Curl

```
curl -X 'POST' \
  'http://localhost:8080/api/auth/login' \
  -H 'accept: */*' \
  -H 'Content-Type: application/json' \
  -d '{"email":"ana.garcia@ecoembes.es","password":"ana123"}'
```

Request URL

```
http://localhost:8080/api/auth/login
```

Server response

Code	Details
200	Response body

```
{
  "token": 1794410661672
}
```

Download

Response headers

```
connection: keep-alive
content-type: application/json
date: Wed,11 Nov 2026 15:24:21 GMT
keep-alive: timeout=60
transfer-encoding: chunked
```

Curl

```
curl -X 'POST' \
  'http://localhost:8080/api/auth/logout?token=1794410661672' \
  -H 'accept: */*' \
  -d ''
```

Request URL

```
http://localhost:8080/api/auth/logout?token=1794410661672
```

Server response

Code	Details
204	Response headers

```
connection: keep-alive
date: Wed,11 Nov 2026 15:26:22 GMT
keep-alive: timeout=60
```

Inicio y cierre de sesión. El servidor devuelve un token al autenticarse y lo invalida al cerrar la sesión

4.3.3. Gestión y consulta de contenedores

Curl

```
curl -X 'POST' \
  'http://localhost:8080/api/containers?token=1794410787428' \
  -H 'accept: */*' \
  -H 'Content-Type: application/json' \
  -d '{
  "id": "CONT-011",
  "address": "Calle de Prueba 11, Bilbao",
  "postalCode": "48009",
  "capacity": 1000
}'
```

Request URL

```
http://localhost:8080/api/containers?token=1794410787428
```

Server response

Code	Details
201	Response body

```
{
  "id": "CONT-011",
  "address": "Calle de Prueba 11, Bilbao",
  "postalCode": "48009",
  "capacity": 1000,
  "packageCount": null,
  "fillLevel": null,
  "estimatedTons": null,
  "readingDate": null
}
```

Download

Response headers

```
connection: keep-alive
content-type: application/json
date: Wed,11 Nov 2026 15:26:43 GMT
keep-alive: timeout=60
transfer-encoding: chunked
```

Curl

```
curl -X 'GET' \
  'http://localhost:8080/api/containers?token=1794410787428' \
  -H 'accept: */*'
```

Request URL

```
http://localhost:8080/api/containers?token=1794410787428
```

Server response

Code	Details
200	Response body

```
[
  {
    "id": "CONT-001",
    "address": "Calle Ercilla 10, Bilbao",
    "postalCode": "48009",
    "capacity": 2000,
    "packageCount": 400,
    "fillLevel": "GREEN",
    "estimatedTons": 0.012,
    "readingDate": "2026-11-11"
  },
  {
    "id": "CONT-002",
    "address": "Gran Vía 25, Bilbao",
    "postalCode": "48009",
    "capacity": 2000,
    "packageCount": 1200,
    "fillLevel": "ORANGE",
    "estimatedTons": 0.036,
    "readingDate": "2026-11-11"
  },
  {
    "id": "CONT-003",
    "address": "Calle Licenciado Poza 30, Bilbao",
    "postalCode": "48011",
    "capacity": 1500,
    "packageCount": 1450,
```

Download

Response headers

```
connection: keep-alive
content-type: application/json
date: Wed,11 Nov 2026 15:26:40 GMT
keep-alive: timeout=60
transfer-encoding: chunked
```

Creación de un contenedor sin lectura inicial y consulta del listado de contenedores

Curl

```
curl -X 'GET' \
  'http://localhost:8080/api/containers/CONT-001/readings?from=2026-11-07&to=2026-11-11&token=1794410787428' \
  -H 'accept: */*'
```

Request URL

```
http://localhost:8080/api/containers/CONT-001/readings?from=2026-11-07&to=2026-11-11&token=1794410787428
```

Server response

Code	Details
200	Response body


```
[
  {
    "date": "2026-11-07",
    "packageCount": 80,
    "fillLevel": "GREEN"
  },
  {
    "date": "2026-11-08",
    "packageCount": 160,
    "fillLevel": "GREEN"
  },
  {
    "date": "2026-11-09",
    "packageCount": 240,
    "fillLevel": "GREEN"
  },
  {
    "date": "2026-11-10",
    "packageCount": 320,
    "fillLevel": "GREEN"
  },
  {
    "date": "2026-11-11",
    "packageCount": 400,
    "fillLevel": "GREEN"
  }
]
```


Download

Response headers

```
connection: keep-alive
content-type: application/json
date: Wed,11 Nov 2026 15:27:18 GMT
keep-alive: timeout=60
transfer-encoding: chunked
```

Curl

```
curl -X 'GET' \
  'http://localhost:8080/api/zones/48009/containers?date=2026-11-11&token=1794410787428' \
  -H 'accept: */*'
```

Request URL

```
http://localhost:8080/api/zones/48009/containers?date=2026-11-11&token=1794410787428
```

Server response

Code	Details
200	Response body


```
[
  {
    "id": "CONT-001",
    "address": "Calle Ercilla 10, Bilbao",
    "postalCode": "48009",
    "capacity": 2000,
    "packageCount": 400,
    "fillLevel": "GREEN",
    "estimatedTons": 0.012,
    "readingDate": "2026-11-11"
  },
  {
    "id": "CONT-002",
    "address": "Gran Vía 25, Bilbao",
    "postalCode": "48009",
    "capacity": 2000,
    "packageCount": 1200,
    "fillLevel": "ORANGE",
    "estimatedTons": 0.036,
    "readingDate": "2026-11-11"
  },
  {
    "id": "CONT-011",
    "address": "Calle de Prueba 11, Bilbao",
    "postalCode": "48009",
    "capacity": 1000,
    "packageCount": 125,
```


Download

Response headers

```
connection: keep-alive
content-type: application/json
date: Wed,11 Nov 2026 15:28:42 GMT
keep-alive: timeout=60
transfer-encoding: chunked
```

Consulta del historial de lecturas y del estado de los contenedores de una zona

4.3.4. Registro diario de lecturas

Curl

```
curl -X 'POST' \
  'http://localhost:8080/api/containers/CONT-011/readings' \
  -H 'accept: */*' \
  -H 'Content-Type: application/json' \
  -d '{
  "packageCount": 125,
  "fillLevel": "GREEN"
}'
```

Request URL

```
http://localhost:8080/api/containers/CONT-011/readings
```

Server response

Code	Details
201	Response body

```
{
  "date": "2026-11-11",
  "packageCount": 125,
  "fillLevel": "GREEN"
}
```

Download

Response headers

```
connection: keep-alive
content-type: application/json
date: Wed,11 Nov 2026 15:27:37 GMT
keep-alive: timeout=60
transfer-encoding: chunked
```

Curl

```
curl -X 'POST' \
  'http://localhost:8080/api/containers/CONT-011/readings' \
  -H 'accept: */*' \
  -H 'Content-Type: application/json' \
  -d '{
  "packageCount": 140,
  "fillLevel": "GREEN"
}'
```

Request URL

```
http://localhost:8080/api/containers/CONT-011/readings
```

Server response

Code	Details
409	Error: response status is 409

Response body

```
{
  "message": "Ya existe una lectura para el contenedor CONT-011 en la fecha 2026-11-11."
}
```

Download

Response headers

```
connection: keep-alive
content-type: application/json
date: Wed,11 Nov 2026 15:27:38 GMT
keep-alive: timeout=60
transfer-encoding: chunked
```

Registro de la primera lectura del día (201) y rechazo de la segunda lectura para el mismo contenedor (409)

Curl

```
curl -X 'GET' \
  'http://localhost:8080/api/containers/CONT-011/readings?from=2026-11-11&to=2026-11-11&token=1794410787428' \
  -H 'accept: */*'
```

Request URL

```
http://localhost:8080/api/containers/CONT-011/readings?from=2026-11-11&to=2026-11-11&token=1794410787428
```

Server response

Code	Details
200	Response body

```
[
  {
    "date": "2026-11-11",
    "packageCount": 125,
    "fillLevel": "GREEN"
  }
]
```

Download

Response headers

```
connection: keep-alive
content-type: application/json
date: Wed,11 Nov 2026 15:28:44 GMT
keep-alive: timeout=60
transfer-encoding: chunked
```

Comprobación posterior: la primera lectura permanece registrada sin modificaciones

4.3.5. Plantas de reciclaje y capacidad

Curl

```
curl -X 'GET' \
  'http://localhost:8080/api/plants?token=1794410787428' \
  -H 'accept: */*'
```

Request URL

```
http://localhost:8080/api/plants?token=1794410787428
```

Server response

Code	Details
200	Response body

```
[
  {
    "id": "CONTSOCKET",
    "name": "ContSocket Ltd."
  },
  {
    "id": "PLASSB",
    "name": "PlasSB Ltd."
  }
]
```

Download

Response headers

```
connection: keep-alive
content-type: application/json
date: Wed,11 Nov 2026 15:28:46 GMT
keep-alive: timeout=60
transfer-encoding: chunked
```

Curl

```
curl -X 'GET' \
  'http://localhost:8080/api/plants/PLASSB/capacity?date=2026-11-11&token=1794410787428' \
  -H 'accept: */*'
```

Request URL

```
http://localhost:8080/api/plants/PLASSB/capacity?date=2026-11-11&token=1794410787428
```

Server response

Code	Details
200	Response body

```
{
  "plantId": "PLASSB",
  "plantName": "PlasSB Ltd.",
  "date": "2026-11-11",
  "totalTons": 0.4,
  "assignedTons": 0,
  "availableTons": 0.4
}
```

Download

Response headers

```
connection: keep-alive
content-type: application/json
date: Wed,11 Nov 2026 15:28:48 GMT
keep-alive: timeout=60
transfer-encoding: chunked
```

Consulta de las plantas disponibles y de la capacidad inicial de PlasSB Ltd.

4.3.6. Asignación y auditoría

Curl

```
curl -X 'POST' \
  'http://localhost:8080/api/plants/PLASSB/assignments?token=1794410787428' \
  -H 'accept: */*' \
  -H 'Content-Type: application/json' \
  -d '{
  "date": "2026-11-11",
  "containerIds": [
    "CONT-003",
    "CONT-006",
    "CONT-009"
  ]
}'
```

Request URL

```
http://localhost:8080/api/plants/PLASSB/assignments?token=1794410787428
```

Server response

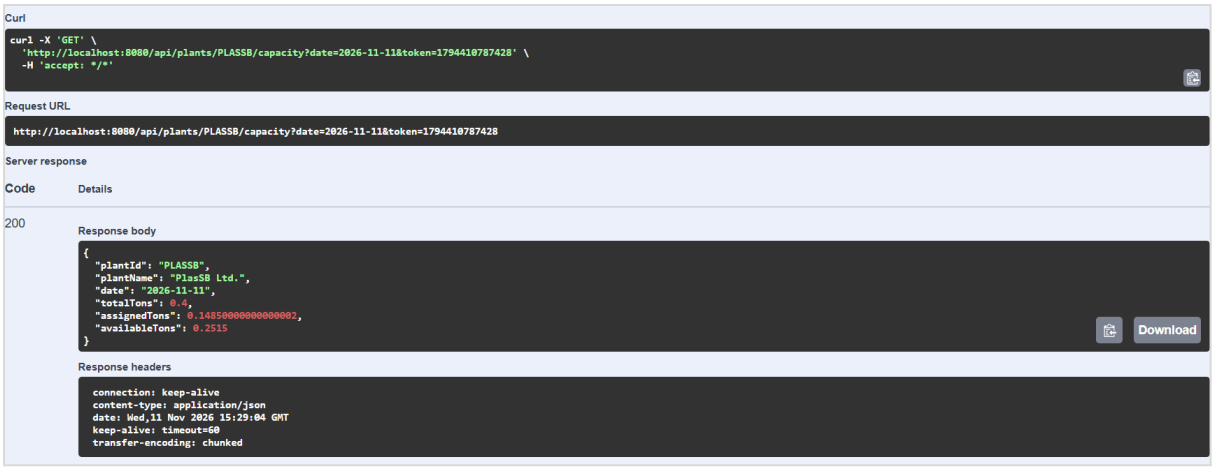
Code	Details
201	Response body

```
{
  "id": 1,
  "plantId": "PLASSB",
  "plantName": "PlasSB Ltd.",
  "date": "2026-11-11",
  "containerIds": [
    "CONT-003",
    "CONT-006",
    "CONT-009"
  ],
  "containerCount": 3,
  "totalPackages": 4950,
  "totalTons": 0.14850000000000002,
  "assignedBy": "ana.garcia@ecoembes.es",
  "createdAt": "2026-11-11T16:29:02.8573991"
}
```

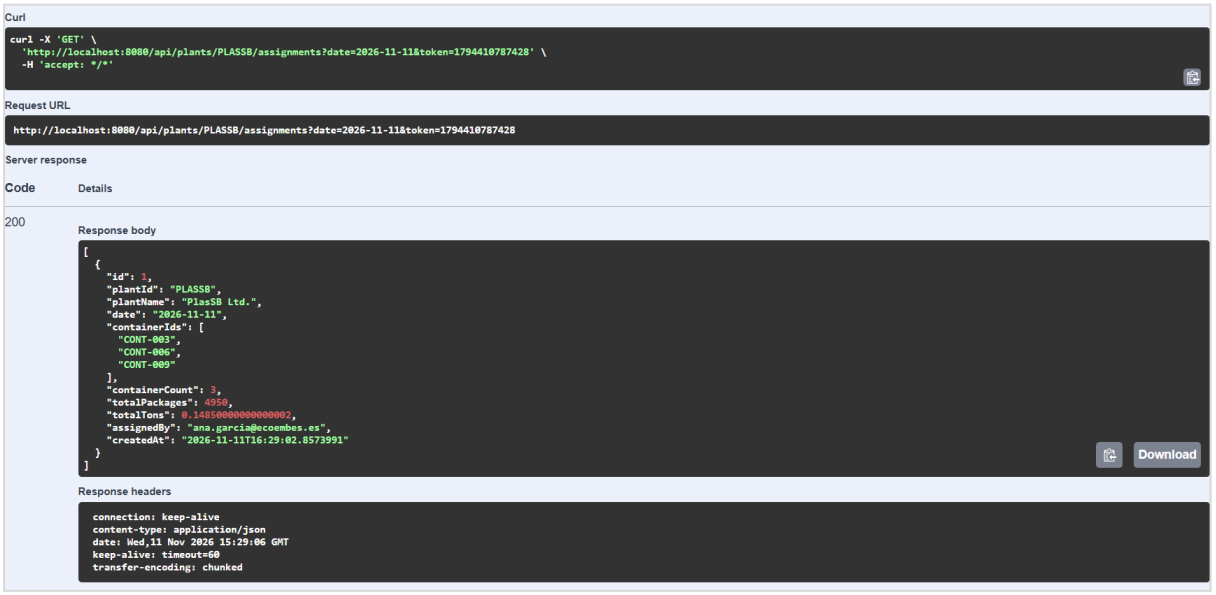
Download

Response headers

```
connection: keep-alive
content-type: application/json
date: Wed,11 Nov 2026 15:29:02 GMT
keep-alive: timeout=60
transfer-encoding: chunked
```

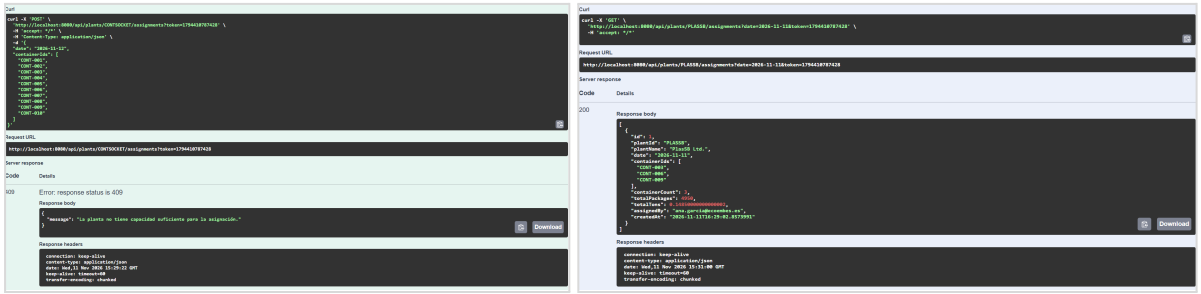


Asignación correcta de tres contenedores y actualización de la capacidad disponible, descontando las 0,1485 toneladas asignadas



Consulta de asignaciones registradas, incluyendo las cantidades y el empleado responsable

4.3.7. Validación de errores y atomicidad



Rechazo de una asignación por capacidad insuficiente (409) y comprobación de que no se han registrado asignaciones parciales